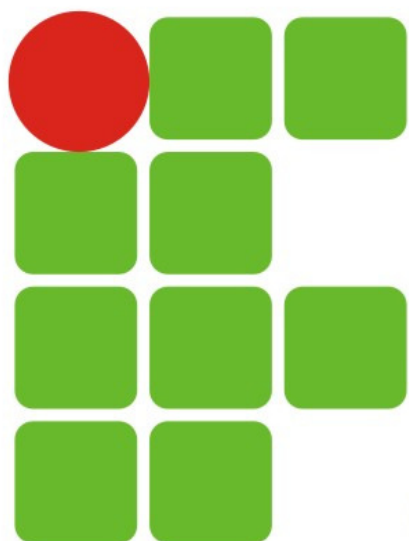




**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
RIO GRANDE DO NORTE

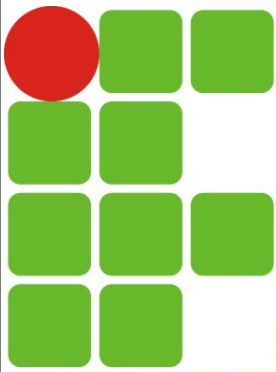


Curso de Tecnologia em Redes de Computadores

Disciplina: Introdução aos Sistemas Abertos

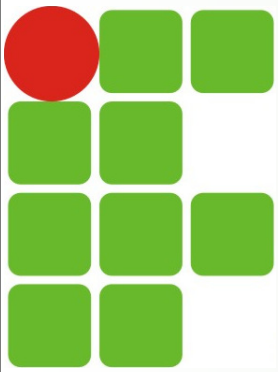
03. Manipulação de arquivos

Prof. Ronaldo <ronaldo.maia@escolar.ifrn.edu.br>



Antes de começar...

- Para melhor fixação do conteúdo, é fundamental que comandos e exemplos apresentados no material e/ou em sala sejam executados
 - Através da máquina virtual Debian 12
 - https://drive.google.com/file/d/1w0S_nndKbX5fT4uEwTO_AC0OeixuDINf/view?usp=sharing
 - Após conclusão do download, no **VirtualBox**, clique no menu "*Arquivo*" => "*Importar Appliance*" e selecione o arquivo *Debian12.ova*
 - Informações da máquina Linux:
 - **Usuário:** sor - **Senha:** sor
 - Ou utilizando um terminal online (outras opções no Classroom, em "Material Auxiliar")
 - <https://bellard.org/jslinux/vm.html?url=alpine-x86.cfg&mem=192&authuser=0>
 - <https://copy.sh/v86/?profile=linux26&authuser=0>

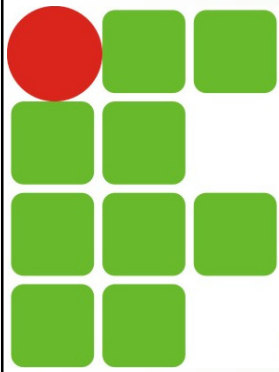


Arquivos e diretórios ocultos

- Na lição anterior, introduzimos a opção `-a` para o comando `ls`, introduzindo os 2 caminhos relativos especiais: `.` e `..`. A opção `-a` inclui também os arquivos e diretórios ocultos. Ex:

```
isa@isa20241:~$ ls -a ~  
.      .bash_history  .bashrc      .profile     .sudo_as_admin_successful  
..     .bash_logout  .cache       .ssh         'teste\1'
```

- Arquivos e diretórios ocultos sempre começam com um ponto (`.`). Por padrão, o diretório home de um usuário inclui diversos arquivos ocultos. Eles servem geralmente para estabelecer as configurações específicas do usuário e devem ser modificados somente por um usuário experiente

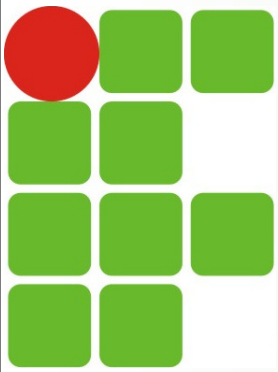


Arquivos e diretórios ocultos

- A opção de lista longa (`ls -l`)
 - Arquivos e diretórios ocuparão uma linha cada, onde serão exibidas informações adicionais sobre cada um. Ex:

```
$ ls -l
-rw-r--r-- 1 user staff      3606 Jan 13  2017 report2018.txt
```

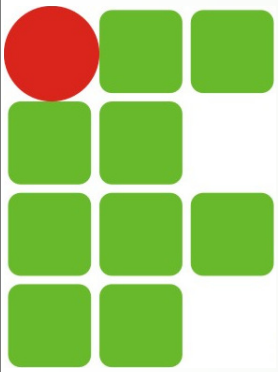
 - `-rw-r--r--` : tipo de arquivo e permissões do arquivo. Note que um arquivo regular começa com traço e um diretório começa com d.
 - `1` : número de links para o arquivo.
 - `user staff` : posse do arquivo. `user` é o proprietário do arquivo, associado ao grupo `staff`
 - `3606` : tamanho do arquivo em bytes
 - `Jan 13 2017` : registro última modificação do arquivo
 - `report2018.txt` : nome do arquivo



Arquivos e diretórios ocultos

■ Opções adicionais do `ls` (principais)

- `ls -lh` : informa tamanhos de arquivo legíveis para humanos: M para Megabytes, K para kilobytes, etc.
- `ls -d */` : lista os diretórios, mas não seu conteúdo. A combinação com `*/` mostra apenas subdiretórios
- `ls -lt` : classifica por data de modificação. Arquivos com as alterações mais recentes aparecem primeiro
- `ls -ltr` : a opção `-r` (reverse) inverte a ordem da classificação (mais antigos primeiro)
- `ls -lx` : classifica por eXtensão de arquivo, agrupando, p ex, os arquivos que terminam com `.txt`, `.jpg`, etc.
- `ls -s` : classifica por tamanho de arquivo (maiores primeiro), e não inclui o conteúdo dos subdiretórios na classificação.
- `ls -R` : exibe uma lista recursiva. O que isso significa?



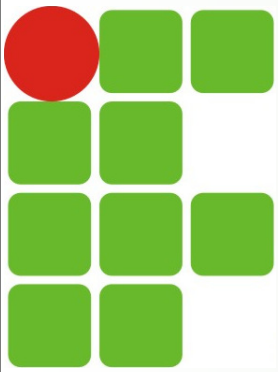
A recursão no bash

- Recursão (ou recursividade) é uma situação em que “algo é definido em termos de si mesmo”. Vamos considerar nosso exemplo de antes:

```
$ ls ~
```

Documents

- Sabemos que `user` possui um diretório inicial (`home`) e que nesse diretório existe um subdiretório. Até agora, o `ls` nos mostrou apenas os arquivos e subdiretórios de um local, sem informações dos subdiretórios. Temos usado `tree` para exibir o conteúdo de vários diretórios. Porém, este não é um dos utilitários integrantes do Linux
- Compare a saída de `tree` com a do `ls -R` nos exemplos a seguir...



A recursão no bash

■ Exemplos:

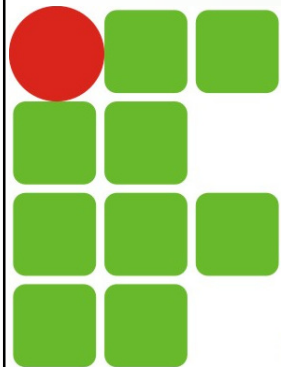
```
$ tree /home/user
user
├── Documents
│   ├── Mission-Statement
│   └── Reports
│       └── report2018.txt
```

```
$ ls -R ~
/home/user/:
Documents

/home/user/Documents:
Mission-Statement
Reports

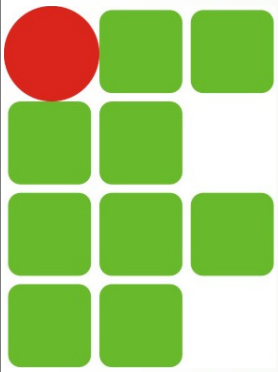
/home/user/Documents/Reports:
report2018.txt
```

- Nota-se na opção recursiva uma lista maior de arquivos, como se rodássemos o `ls` no diretório inicial de `user`, e a cada subdiretório fosse executado o `ls` novamente
 - A recursividade é particularmente importante nos comandos de modificação de arquivos, como copiar ou remover diretórios
 - Ex.: para copiar o subdiretório `Documents`, deve se especificar uma cópia recursiva para estender esse comando a todos os subdiretórios



Criando, movendo e deletando arquivos

- Um arquivo é uma coleção de dados com nome e conjunto de atributos
- Um diretório é um tipo especial de arquivo usado para organizar arquivos
 - Podemos compará-los às pastas suspensas usadas para organizar papéis em um gaveteiro metálico, porém, com uma maior facilidade de colocar diretórios dentro de outros
- A linha de comando é a forma mais eficaz de gerenciar arquivos no Linux
 - Suas ferramentas têm recursos que tornam a CLI mais rápida e fácil do que um gerenciador de arquivos gráfico
 - Nesta seção, usaremos os comandos **ls**, **cd**, **mkdir**, **touch**, **cat**, **echo**, **mv**, **rmdir**, **rm** e **cp** para visualizar, gerenciar e organizar arquivos e diretórios



cd

■ Muda de diretório

■ Sintaxe: **cd** [diretório]

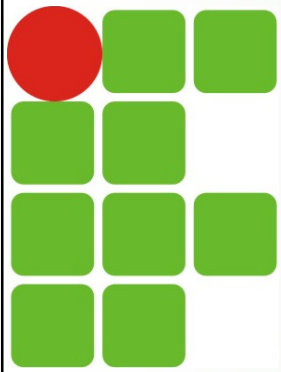
■ Exemplos:

```
user@debian:~/aula$ cd /tmp
```

```
user@debian:/tmp$ cd
```

```
user@debian:~$ cd aula
```

```
user@debian:~/aula$ cd ..
```



mkdir

■ Cria diretórios

■ Sintaxe: **mkdir** [opções] <diretório(s)>

■ Opções (principais):

■ **-p**: Cria os diretórios pais

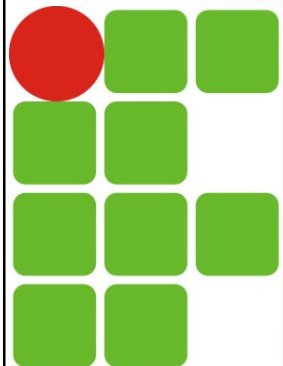
■ Exemplos:

```
user@debian:~/aula$ mkdir teste
```

```
user@debian:~/aula$ mkdir TMS TII TADM
```

```
user@debian:~/aula$ mkdir -p avo/pai/filho/neto
```

```
user@debian:~/aula$ mkdir ISA_{aulas,exer,provas}
```

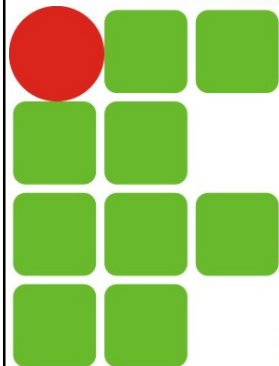


mkdir

■ Exemplo (a ser seguido durante a lição):

```
$ mkdir creating moving copying/files copying/directories deleting/directories
deleting/files globs
mkdir: cannot create directory 'copying/files': No such file or directory
mkdir: cannot create directory 'copying/directories': No such file or directory
mkdir: cannot create directory 'deleting/directories': No such file or directory
mkdir: cannot create directory 'deleting/files': No such file or directory
$ ls
creating globs moving
```

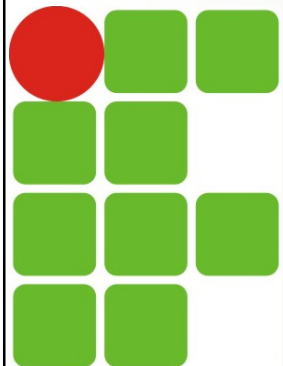
- Apenas os diretórios `moving`, `globs` e `creating` foram criados, pois `copying` e `deleting` ainda não existem
- O `mkdir`, por padrão, não cria um diretório dentro de um diretório que não existe. Para resolver esta situação, basta utilizar a opção `-p` ou `--parents`



touch

- Cria arquivo(s) vazio(s) ou altera o registro de data e hora (*timestamp*) de arquivos existentes
 - Sintaxe: **touch** [opções] <arquivo(s)>
 - Exemplos (a serem usados durante a aula)

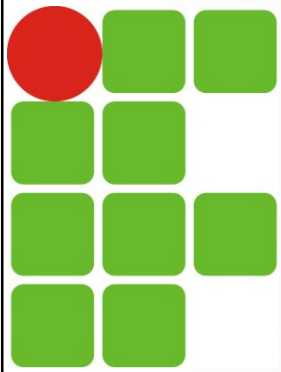
```
$ touch globs/question1 globs/question2012 globs/question23 globs/question13  
globs/question14  
$ touch alobs/star10 alobs/star1100 alobs/star2002 alobs/star2013  
$ cd globs  
$ ls  
question1    question14    question23    star1100    star2013  
question13    question2012    star10        star2002  
$ echo hello > question15  
$ cat question15  
hello
```



touch

- Também é possível criar múltiplos arquivos em sequência e/ou intervalos usando `{ .. }`. Exemplos:

```
isa@isa20241:~$ touch teste{1..4}
isa@isa20241:~$ ls
teste1 teste2 teste3 teste4
isa@isa20241:~$ touch teste_{a..d}.{txt,html}
isa@isa20241:~$ ls
teste1 teste3 teste_a.html teste_b.html teste_c.html teste_d.html
teste2 teste4 teste_a.txt teste_b.txt teste_c.txt teste_d.txt
isa@isa20241:~$ touch teste_{1..4,8,isa}.exe
isa@isa20241:~$ ls
teste1      teste3      teste_a.html teste_b.txt  teste_d.html
teste_1..4.exe teste4      teste_a.txt  teste_c.html teste_d.txt
teste2      teste_8.exe teste_b.html teste_c.txt  teste_isa.exe
isa@isa20241:~$
```



cat

- Concatena arquivos e mostra conteúdo na saída padrão

- Sintaxe: **cat** [arquivo(s)]

- Exemplos:

```
user@debian:~/aula$ cat config.txt
```

```
user@debian:~/aula$ cat a1.txt a2.txt a3.txt > a4.txt
```

```
user@debian:~/aula$ cat > novoarq.txt
```

```
.
```

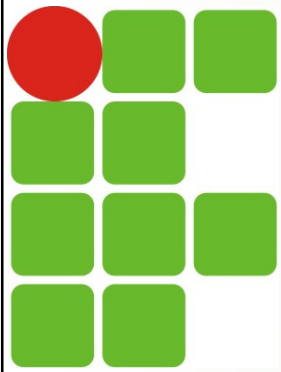
```
(digite algo)
```

```
.
```

```
Ctrl+d (ou Ctrl+c)
```

- **tac**

- ?



echo

■ Escreve no terminal ou em um arquivo

■ Sintaxe: **echo** [opções] <texto>

■ Opções (principais):

- -n: Não faz a quebra de linha após texto
- -e: Habilita a interpretação de caracteres especiais
- -E: Desabilita a interpretação de caracteres especiais (padrão)

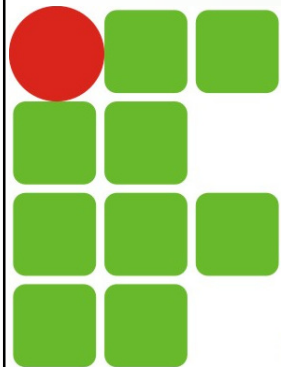
■ Exemplos:

```
user@debian:~/aula$ echo "Olá mundo"
```

```
user@debian:~/aula$ echo -n "Olá mundo"
```

```
user@debian:~/aula$ echo -e "Olá \n mundo"
```

```
user@debian:~/aula$ echo -e "Olá \n mundo" > ola.txt
```



mv

■ Move ou renomeia arquivos

■ Sintaxes:

■ **mv** [opções] <arquivo-antigo> <arquivo-novo>

■ **mv** [opções] <arquivo> <diretório>

■ Opções (principais):

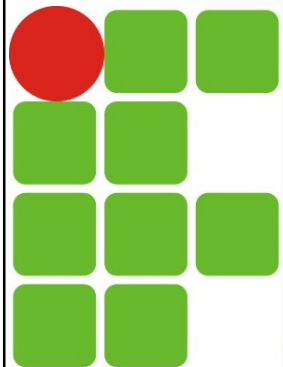
■ **-i**: Pergunta antes de sobrescrever

■ **-f**: Não solicita nenhuma confirmação

■ Exemplos:

```
user@debian:~/aula$ mv prog1.c exercicio.c
```

```
user@debian:~/aula$ mv *.c /home/aluno/programas/
```

rmmdir

■ Apaga diretórios vazios

■ Sintaxe: **rmmdir** [opções] <diretório(s)>

■ Opções (principais):

■ **-p**: remove a árvore de diretórios (iniciando da direita para a esquerda), contanto que estejam vazias

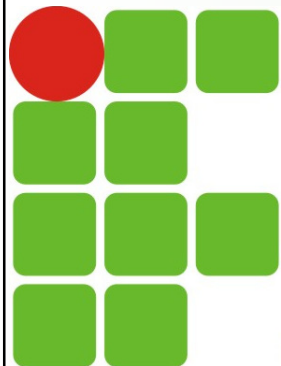
■ Exemplos:

```
user@debian:~/aula$ rmmdir teste
```

```
user@debian:~/aula$ rmmdir TMS TII TADM
```

```
user@debian:~/aula$ rmmdir avo/pai/filho/neto
```

```
user@debian:~/aula$ rmmdir -p avo/pai/filho/neto
```



rm

■ Apaga arquivos

■ Sintaxe: **rm** [opções] <arquivo(s)>

■ Opções (principais):

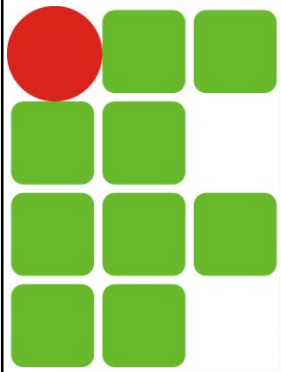
- **-i**: Pergunta antes de apagar
- **-f, --force**: Apaga sem solicitar confirmação
- **-r, -R, --recursive**: Apaga árvore de **diretórios** recursivamente

■ Exemplos:

```
user@debian:~/aula$ rm prog1.c prog1-backup.c
```

```
user@debian:~/aula$ rm -i documentos/*
```

```
user@debian:~/aula$ rm -rf temp/
```



cp

■ Cópia arquivos

■ Sintaxe: **cp** [opções] <arq-orig> <arq-dest>

■ Opções (principais):

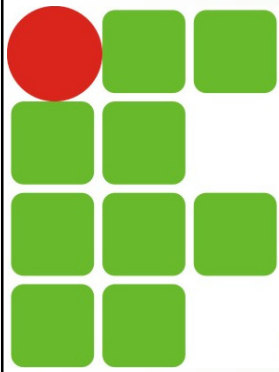
- **-i**: Pergunta antes de sobrescrever arquivos existentes
- **-p**: Preserva permissões, proprietários e datas
- **-r, -R, --recursive**: Cópia recursivamente

■ Exemplos:

```
user@debian:~/aula$ cp prog1.c prog1-backup.c
```

```
user@debian:~/aula$ cp *.c /home/aluno/programas
```

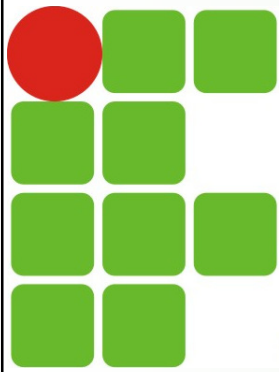
```
user@debian:~/aula$ cp teste.doc tela.jpg /tmp
```



Globbing

- Costuma-se chamar de *globbing* uma linguagem simples de correspondência de padrões. Os shells nos sistemas Linux usam essa linguagem para buscar grupos de arquivos cujos nomes correspondam a um padrão específico (POSIX.1-2017*), com os seguintes padrões para a correspondência de caracteres:
 - * : Corresponde a qualquer número de quaisquer caracteres, incluindo nenhum caractere
 - ? : Corresponde a exatamente um caractere qualquer
 - [] : Usados para combinar faixas ou classes de caracteres

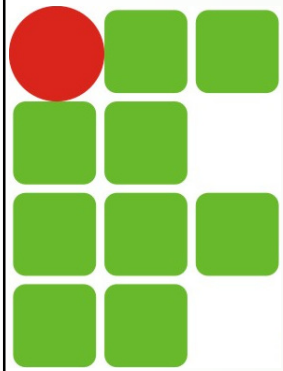
OBS*: O padrão atual é o **IEEE 1003.1-2024** (*IEEE/Open Group Standard for Information Technology--Portable Operating System Interface (POSIX™) Base Specifications, Issue 8*), publicado em 14/06/2024 - https://store.accuristech.com/ieee/standards/ieee-1003-1-2024?product_id=2562767&vendor_id=7700



Globbing

- Resumindo, o shell pode procurar por um padrão, em vez de uma sequência literal de texto. Os usuários Linux especificam diversos arquivos com um glob em vez de digitar o nome de cada arquivo.
Ex:

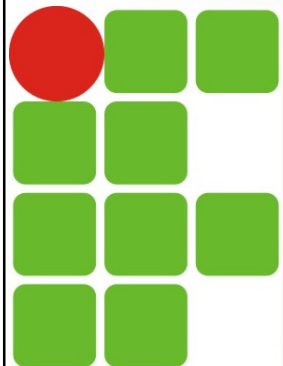
```
$ cd ~/linux_essentials-2.4/globs
$ ls
question1    question14  question2012  star10      star2002
question13   question15  question23    star1100    star2013
$ ls star1*
star10  star1100
$ ls star*
star10  star1100  star2002  star2013
$ ls star2*
star2002  star2013
$ ls star2*2
star2002
$ ls star2013*
star2013
```



Globbing

- O `?` expande para qualquer caractere único. Experimente os seguintes comandos do exemplo abaixo:

```
$ ls
question1    question14  question2012  star10      star2002
question13   question15  question23    star1100    star2013
$ ls question?
question1
$ ls question1?
question13  question14  question15
$ ls question?3
question13  question23
$ ls question13?
ls: cannot access question13?: No such file or directory
```



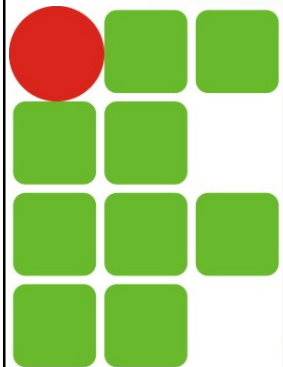
Globbing

- Os colchetes `[]` são usados para combinar faixas ou classes de caracteres. Crie alguns arquivos para testar

```
$ mkdir brackets
$ cd brackets
$ touch file1 file2 file3 file4 filea fileb filec file5 file6 file7
```

- Os intervalos entre `[]` são expressos usando um `-`:

```
$ ls
file1 file2 file3 file4 file5 file6 file7 filea fileb filec
$ ls file[1-2]
file1 file2
$ ls file[1-3]
file1 file2 file3
```



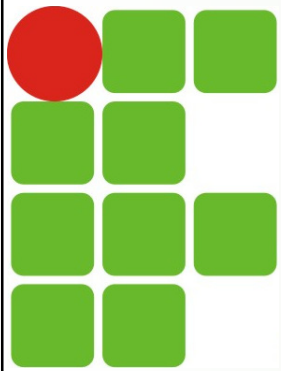
Globbing

- É possível especificar vários intervalos:

```
$ ls file[1-25-7]
file1  file2  file5  file6  file7
$ ls file[1-35-6a-c]
file1  file2  file3  file5  file6  filea  fileb  filec
```

- Os colchetes também podem ser usados para encontrar um conjunto específico de caracteres correspondentes

```
$ ls file[1a5]
file1  file5  filea
```

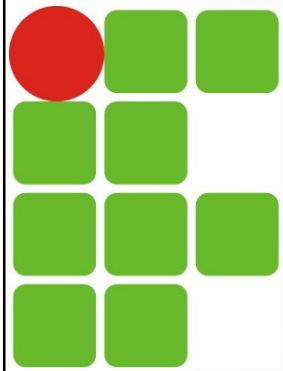
Globbing

- Os `[]` podem conter `^` precedendo os caracteres que **NÃO** devem corresponder à pesquisa. Exemplo:

```
$ ls file[^a]
file1 file2 file3 file4 file5 file6 file7 fileb filec
```

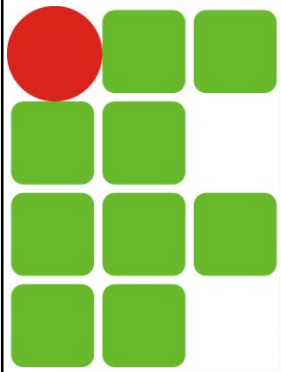
- Os colchetes também podem definir **classes de caracteres** `[ :classname: ]`. Assim, para fazermos a correspondência com números (classe `digit`), poderíamos fazer:

```
$ ls file[[:digit:]]
file1 file2 file3 file4 file5 file6 file7
$ touch file1a file11
$ ls file[[:digit:]]a
file1 file2 file3 file4 file5 file6 file7 filea
$ ls file[[:digit:]]a
file1a
```



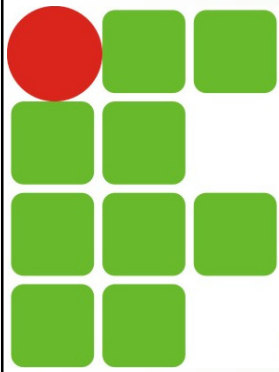
Globbing

- **Classes de caracteres** definidas pelo padrão POSIX:
 - `[ :alnum: ]` - Letras e números
 - `[ :alpha: ]` - Maiúsculas ou minúsculas
 - `[ :blank: ]` - Espaços e tabulações
 - `[ :cntrl: ]` - Caracteres de controle, como backspace, bell, confirmação negativa, escape
 - `[ :digit: ]` - Numerais (0123456789)
 - `[ :graph: ]` - Caracteres gráficos (todos os caracteres, exceto ctrl e o caractere de espaço)
 - `[ :lower: ]` - Letras minúsculas (a-z)



Globbing

- **Classes de caracteres** definidas pelo padrão POSIX:
 - `[:print:]` - Caracteres imprimíveis (alnum, punct e o caractere de espaço)
 - `[:punct:]` - Caracteres de pontuação, ou seja, !, &, "
 - `[:space:]` - Caracteres de espaço em branco, por exemplo, tabulações, espaços, novas linhas
 - `[:upper:]` - Letras maiúsculas (A-Z)
 - `[:xdigit:]` - Números hexadecimais (geralmente 0123456789abcdefABCDEF)



Referências

- LPI Linux Essentials
 - Tópico 2: Encontrando seu caminho em um Sistema Linux (Seções 2.3 e 2.4)
<https://learning.lpi.org/pt/learning-materials/010-160/?authuser=0>
- Material de aula do prof. Carlos Gustavo
- <https://www.hostinger.com.br/tutoriais/comando-touch-linux>
- [https://store accuristech.com/ieee/standards/ieee-1003-1-2024?vendor id=7700&product id=2562767](https://store accuristech.com/ieee/standards/ieee-1003-1-2024?vendor_id=7700&product_id=2562767)